# MCQs Finite State Machine (FSM) and Verilog Code

**1. What is the main function of a divide-by-3 FSM?**

- A. To multiply the clock frequency by 3
- B. To generate an output frequency equal to one-third of the input clock frequency
- C. To divide the input voltage by 3
- D. To generate three independent clock signals

**Answer:** B

**Explanation:** A divide-by-3 circuit produces an output frequency f_out = f_in / 3. For example, a 30 MHz input produces a 10 MHz output.

**2. How many states are required for a basic divide-by-3 FSM?**

- A. 1
- B. 2
- C. 3
- D. 4

**Answer:** C

**Explanation:** The FSM cycles through three states: S0 → S1 → S2 → S0. Therefore, three states are required.

**3. What is the next state after S2 in a divide-by-3 FSM?**

- A. S0
- B. S1
- C. S2
- D. S3

**Answer:** A

**Explanation:** The state sequence is cyclic: S0 → S1 → S2 → S0. After S2, the FSM returns to S0.

**4. What is the minimum number of bits required to encode three FSM states?**

- A. 1 bit
- B. 2 bits

- C. 3 bits
- D. 4 bits

**Answer:** B

**Explanation:** One bit represents only two states, while two bits represent four states. Thus, two bits are sufficient for three states.

**5. Which Verilog statement is commonly used to update the state at the rising edge of the clock?**

- A. always @(*)
- B. always @(posedge clk)
- C. always @(clk)
- D. always @(negedge reset)

**Answer:** B

**Explanation:** always @(posedge clk) executes when the clock changes from 0 to 1. FSM state registers are commonly updated at this edge.

**6. What is the purpose of the reset signal in a divide-by-3 FSM?**

- A. To increase the clock frequency
- B. To initialize the FSM to a known state
- C. To generate an output pulse
- D. To add a new state

**Answer:** B

**Explanation:** Reset initializes the FSM to a known starting state, usually S0, preventing an unknown starting condition.

**7. Which Verilog code correctly describes the state transitions?**

- A. S0→S1, S1→S2, S2→S0
- B. S0→S0, S1→S1, S2→S2
- C. S0→S2, S1→S0, S2→S1
- D. S0→S3, S1→S4, S2→S5

**Answer:** A

**Explanation:** The correct divide-by-3 sequence is S0 → S1 → S2 → S0, repeating every three clock cycles.

**8. What is the purpose of next_state in a two-process FSM?**

- A. It stores the previous state
- B. It represents the state to be loaded at the next clock edge
- C. It stores the output frequency
- D. It acts as the reset signal

**Answer:** B

**Explanation:** next_state is calculated by combinational logic. At the next active clock edge, it is copied into the state register.

**9. Why is a default case included in FSM transition logic?**

- A. To increase the number of states
- B. To handle invalid or undefined states safely
- C. To increase the output frequency
- D. To remove the need for reset

**Answer:** B

**Explanation:** A default case provides a recovery path. For example, default: next_state = S0; returns the FSM to a known state.

**10. If the present state is S1 in the following sequence S0→S1→S2→S0, what is the next state?**

- A. S0
- B. S1
- C. S2
- D. Unknown

**Answer:** C

**Explanation:** According to the transition sequence, S1 is followed by S2. The FSM enters S2 at the next active clock edge.

### Sample Verilog Code: Divide-by-3 FSM

```
module divide_by_3_fsm (
    input  wire clk,
    input  wire reset,
    output reg  div_out
);

    localparam S0 = 2'b00,
               S1 = 2'b01,
               S2 = 2'b10;

    reg [1:0] state, next_state;

    // State register
    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= S0;
        else
            state <= next_state;
    end

    // Next-state logic
    always @(*) begin
        case (state)
            S0: next_state = S1;
            S1: next_state = S2;
            S2: next_state = S0;
            default: next_state = S0;
        endcase
    end

    // Output logic
    always @(*) begin
        if (state == S0)
            div_out = 1'b1;
        else
            div_out = 1'b0;
    end

endmodule
```

Note: This simple FSM repeats every three clock cycles. A true 50% duty-cycle divide-by-3 clock generally requires additional logic using both clock edges.

## MCQs on Moore Pattern Recognizer FSM and Verilog Code

**Pattern used:** 1011 (overlapping pattern detection). In a Moore FSM, the output depends only on the present state.

**11. What is the main purpose of a pattern recognizer FSM?**

- A. To increase clock frequency
- B. To detect a specified sequence of input bits
- C. To divide the input voltage
- D. To store unlimited data

**Answer:** B

**Explanation:** A pattern recognizer FSM monitors a serial input stream and asserts an output when the required bit pattern is detected.

**12. In a Moore FSM, the output depends on:**

- A. Only the present state
- B. Only the current input
- C. Both present state and current input
- D. The clock frequency only

**Answer:** A

**Explanation:** In a Moore machine, outputs are determined solely by the current state. The input affects the next state, not the output directly.

**13. For recognizing the pattern 1011, which state usually produces output 1?**

- A. Initial state
- B. State representing that no bits are matched
- C. Final detection state
- D. Any state when input is 1

**Answer:** C

**Explanation:** A dedicated final state represents successful recognition of 1011. The Moore output is asserted in this detection state.

**14. How many states are commonly used for a Moore FSM that recognizes 1011?**

- A. 2

- B. 3
- C. 4
- D. 5

**Answer:** D

**Explanation:** A typical Moore recognizer uses five states: no match, matched 1, matched 10, matched 101, and matched 1011.

**15. What is the correct progression when the input sequence 1, 0, 1, 1 is received?**

- A. S0 → S1 → S2 → S3 → S4
- B. S0 → S2 → S1 → S4 → S3
- C. S0 → S1 → S1 → S1 → S1
- D. S0 → S4 → S3 → S2 → S1

**Answer:** A

**Explanation:** Each state represents the portion of 1011 matched so far. After receiving all four bits, the FSM reaches S4, the detection state.

**16. Which Verilog construct is commonly used to describe next-state transitions?**

- A. case
- B. assign only
- C. initial
- D. generate only

**Answer:** A

**Explanation:** A case statement clearly maps each present state and input condition to the appropriate next state.

**17. Why is a separate output block used in a Moore FSM?**

- A. To make output depend directly on input
- B. To generate output only from the present state
- C. To remove the clock
- D. To avoid defining states

**Answer:** B

**Explanation:** Moore output logic is based only on the current state. For example, detect = 1 when state is S4; otherwise detect = 0.

**18. What is the purpose of the reset signal?**

- A. To force the FSM into a known initial state
- B. To detect the pattern immediately
- C. To change 0 into 1
- D. To increase the number of states

**Answer:** A

**Explanation:** Reset initializes the state register, typically to S0, so pattern recognition begins from a known condition.

**19. What does overlapping detection mean for the pattern 1011?**

- A. The FSM ignores repeated bits
- B. A bit from one detected pattern may also be used as the start of another pattern
- C. The pattern can be detected only once
- D. The input must be reset after every bit

**Answer:** B

**Explanation:** Overlapping detection preserves useful suffix bits. For example, after detecting 1011, the final 1 can serve as the first 1 of a possible new pattern.

**20. Why is a default case included in the next-state logic?**

- A. To increase the output frequency
- B. To handle invalid or undefined states safely
- C. To make the output depend on input
- D. To eliminate reset

**Answer:** B

**Explanation:** A default transition, such as next_state = S0, provides a safe recovery path if the FSM enters an unexpected state.

## Sample Verilog Code: Moore Pattern Recognizer for 1011

```
module moore_pattern_1011 (
    input  wire clk,
    input  wire reset,
    input  wire x,
    output reg  detect
);

    localparam S0 = 3'b000, // No match
               S1 = 3'b001, // Matched: 1
               S2 = 3'b010, // Matched: 10
               S3 = 3'b011, // Matched: 101
               S4 = 3'b100; // Matched: 1011

    reg [2:0] state, next_state;

    // State register
    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= S0;
        else
            state <= next_state;
    end

    // Next-state logic
    always @(*) begin
        case (state)
            S0: begin
                if (x) next_state = S1;
                else   next_state = S0;
            end
            S1: begin
                if (x) next_state = S1;
                else   next_state = S2;
            end
            S2: begin
                if (x) next_state = S3;
                else   next_state = S0;
            end
            S3: begin
                if (x) next_state = S4;
                else   next_state = S2;
            end
            S4: begin
                if (x) next_state = S1;
                else   next_state = S2;
            end
            default: next_state = S0;
        endcase
    end

    // Moore output logic
    always @(*) begin
        if (state == S4)
            detect = 1'b1;
        else
            detect = 1'b0;
```

```
    end

endmodule
```

Note: The code detects the pattern 1011 with overlapping recognition. Since this is a Moore FSM, detect becomes 1 only after the FSM enters the final detection state S4.

# MCQs on Mealy Pattern Recognizer FSM and Verilog Code

**Pattern used:** 1011 (overlapping pattern detection). In a Mealy FSM, the output depends on both the present state and the current input.

**21. What is the main purpose of a Mealy pattern recognizer FSM?**

- A. To multiply the clock frequency
- B. To detect a specified sequence in a serial input stream
- C. To store all input bits permanently
- D. To divide the input voltage

**Answer:** B

**Explanation:** A pattern recognizer monitors incoming bits and produces an output when the specified sequence is detected.

**22. In a Mealy FSM, the output depends on:**

- A. Only the present state
- B. Only the current input
- C. Both the present state and the current input
- D. Only the reset signal

**Answer:** C

**Explanation:** In a Mealy machine, output is a function of the present state and the current input. Therefore, the output may change immediately when the input changes.

**23. For detecting the pattern 1011, how many states are commonly required in a Mealy FSM?**

- A. 2
- B. 3
- C. 4
- D. 5

**Answer:** C

**Explanation:** A typical Mealy recognizer uses four states representing: no match, matched 1, matched 10, and matched 101. Detection occurs on the transition caused by the final input 1.

**24. In a Mealy recognizer for 1011, when is detect asserted?**

- A. Only after entering a separate final state
- B. When the FSM is in the matched-101 state and the current input is 1
- C. Whenever the input is 1
- D. Only during reset

**Answer:** B

**Explanation:** When the previous bits have matched 101 and the current input is 1, the complete pattern 1011 is recognized. The output is asserted on that transition.

**25. Which statement best describes a key difference between Mealy and Moore FSMs?**

- A. Mealy FSMs have no states
- B. Moore outputs depend on input and state, while Mealy outputs depend only on state
- C. Mealy outputs depend on state and input, while Moore outputs depend only on state
- D. Both always use the same number of states

**Answer:** C

**Explanation:** A Mealy output uses both state and input. A Moore output uses only the present state. Mealy machines often require fewer states for the same pattern.

**26. Which Verilog construct is commonly used to describe state transitions?**

- A. case

- B. initial only
- C. generate only
- D. task only

**Answer:** A

**Explanation:** A case statement is commonly used to describe the transition rules for each present state and input value.

**27. What is the purpose of next_state in a two-process FSM design?**

- A. It stores the output permanently
- B. It represents the state to be loaded at the next clock edge
- C. It replaces the input signal
- D. It acts as the reset signal

**Answer:** B

**Explanation:** Combinational logic calculates next_state. The state register loads next_state on the active clock edge.

**28. What does overlapping pattern detection mean?**

- A. A detected pattern cannot share bits with another pattern
- B. A bit from one detected pattern can also be part of the next pattern
- C. The input is ignored after detection
- D. The FSM resets after every input bit

**Answer:** B

**Explanation:** Overlapping detection allows useful suffix bits to become the beginning of another possible pattern. This improves continuous pattern recognition.

**29. What is the purpose of the reset signal?**

- A. To initialize the FSM to a known state
- B. To force detect permanently high
- C. To add a new state
- D. To increase input frequency

**Answer:** A

**Explanation:** Reset places the FSM in the initial no-match state, ensuring predictable operation.

**30. Why is a default case included in the FSM logic?**

- A. To increase the number of states
- B. To handle invalid or undefined states safely
- C. To make the clock faster
- D. To remove the input signal

**Answer:** B

**Explanation:** A default assignment provides a safe recovery path, usually returning the FSM to the initial state.

## Sample Verilog Code: Mealy Pattern Recognizer for 1011

```
module mealy_pattern_1011 (
    input  wire clk,
    input  wire reset,
    input  wire x,
    output reg  detect
);

    localparam S0 = 2'b00, // No match
               S1 = 2'b01, // Matched: 1
               S2 = 2'b10, // Matched: 10
               S3 = 2'b11; // Matched: 101

    reg [1:0] state, next_state;

    // State register
    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= S0;
        else
            state <= next_state;
    end

    // Next-state and Mealy output logic
    always @(*) begin
        next_state = S0;
        detect = 1'b0;

        case (state)
            S0: begin
                if (x)
                    next_state = S1;
                else
                    next_state = S0;
            end
```

```
            S1: begin
                if (x)
                    next_state = S1;
                else
                    next_state = S2;
            end

            S2: begin
                if (x)
                    next_state = S3;
                else
                    next_state = S0;
            end

            S3: begin
                if (x) begin
                    detect = 1'b1;
                    next_state = S1;
                end
                else begin
                    next_state = S2;
                end
            end

            default: begin
                next_state = S0;
                detect = 1'b0;
            end
        endcase
    end

endmodule
```

Note: This Verilog design detects the pattern 1011 with overlapping recognition. In the S3 state, receiving x = 1 completes the pattern and immediately asserts detect = 1. The next state is S1 because the final 1 can also be the first bit of a new pattern.